

React (with Vite & Tailwind) – From Basics to Advanced

Introduction to React: React is a **JavaScript library for building user interfaces** [21†L232-L240] . It lets you create reusable UI *components* – self-contained pieces of code that describe a portion of the UI [21†L243-L247] . You compose these components to build complex interfaces. Under the hood, React uses a **virtual DOM** (an in-memory UI representation) to efficiently update the real DOM only where changes occur [7†L155-L163] [21†L243-L247] . This makes UI updates fast and smooth. The typical React component is a JavaScript function returning JSX (an HTML-like syntax) [4†L109-L118] . For example:

```
function MyButton() {  
  return <button>Click me</button>;  
}  
export default MyButton;
```

Here `MyButton` is a React component (note the capitalized name) that returns a `<button>` element. You render it inside other components (e.g. `<MyButton />`) [4†L109-L118] .

JSX: JSX lets you write HTML-like markup in JavaScript. It is stricter than plain HTML: tags must be closed (e.g. `<br />`), and if you return multiple elements they must be wrapped in a single parent (e.g. `<div>...</div>`) [4†L179-L187] . For example:

```
function AboutPage() {  
  return (  
    <div>  
      <h1>About</h1>  
      <p>Hello there.</p>  
    </div>  
  );  
}
```

Notice all tags are closed and `<h1>` and `<p>` are inside one `<div>` [4†L179-L187] .

DOM vs Virtual DOM: In browsers, the **Real DOM** is a tree representation of all HTML elements. Directly updating the real DOM is slow for large UIs. React's **Virtual DOM** is a lightweight in-memory copy of the UI. When state changes, React diffs the new virtual DOM against the old one and updates only the changed parts of the real DOM [7†L155-L163] . As freeCodeCamp explains, “the virtual DOM uses a diff algorithm to compare the current and previous versions... It only re-renders the parts of the UI that have changed, instead of the whole thing” [7†L163-L170] . This makes React efficient: you describe *what* the UI should look like, and React takes care of updating the actual DOM.

Setting Up React with Vite

To start a React project, you need a build tool. The modern recommendation is [Vite](#) – a lightning-fast frontend build tool. React's official docs say: *"The first step is to install a build tool like vite ... These build tools provide features to package and run source code, provide a development server for local development and a build command for production"* [1†L133-L142] . Vite comes with sensible defaults and supports JSX out of the box.

Creating a Vite-React project: In your terminal, run:

```
npm create vite@latest my-app -- --template react
```

This scaffolds a new React app. (Replace `my-app` with your project name. The `react-ts` template can be used if you want TypeScript [1†L149-L154] .) This sets up a project with a ready-to-go `index.html` and entry files. For example, `vite.config.js` will already be configured, and an `index.html` will load your React code. After creation, install dependencies (`npm install`) and start the dev server (`npm run dev`). Vite will serve your app at `http://localhost:5173` by default. Its docs note that "`index.html` is front-and-center ... Vite treats `index.html` as source code and part of the module graph," making setup very straightforward [18†L389-L398] .

Using npm scripts: A typical Vite React project's `package.json` includes scripts like:

```
"scripts": {
  "dev": "vite",          // starts the dev server
  "build": "vite build",  // builds for production
  "preview": "vite preview" // preview production build
}
```

Run `npm run dev` to launch the app and `npm run build` to bundle it for production [18†L418-L427] .

Citation: Official React docs on using Vite and build tools [1†L133-L142] [1†L149-L154] .

Components in React

React apps are **built from components**. Each component is a self-contained function (or class) that returns some UI. Most modern React uses *functional components*. For example:

```
function WelcomeBanner() {
  return <h2>Welcome to my app!</h2>;
}
```

Components can be composed: you can nest one inside another. For instance, in `App.js` you might use our `WelcomeBanner` :

```
export default function App() {
  return (
    <div>
      <h1>Main App</h1>
      <WelcomeBanner />
    </div>
  );
}
```

Here `<WelcomeBanner />` renders inside `<App>` [4†L125-L133] . Notice two things: component names must start with a capital letter, and the JSX tag `<WelcomeBanner />` has the same name as our function [4†L139-L145] . HTML tags are lowercase, React components are uppercase.

Files and Exports: It's common to put each component in its own file. You export it from that file and import it elsewhere. For example, in `Profile.js` :

```
// Profile.js
export default function Profile() {
  return ;
}
```

Then in `App.js` :

```
import Profile from './Profile.js';

export default function App() {
  return <div><Profile /></div>;
}
```

React docs explain this modular approach: move components into separate files, **export** them (default or named), then **import** where needed [5†L156-L164] . This keeps code organized and components reusable.

JSX rules: In JSX, use `className` instead of `class` for CSS classes. For example: `<div className="container">Hello</div>` . React specifically requires `className` [4†L209-L213] . Also, JSX expressions can use JavaScript in curly braces. For example, to include a variable: `<span>{userName}</span>` .

Key points: Components are the building blocks. Use functions, start names uppercase, return JSX. Export/import to split code [5†L156-L164] .

Example component with props (next sections will explain props further):

```
function Greeting({ name }) {
  return <h3>Hello, {name}!</h3>;
}
```

```
export default Greeting;  
// Usage: <Greeting name="Alice" />
```

This demonstrates a component (`Greeting`) that receives a prop `name` and renders it.

Props (Component Inputs)

What are props? *Props* (properties) are inputs passed to components. They are analogous to function arguments. Parents pass data to children via props. React docs say “React components use props to communicate with each other. Every parent component can pass information to its child components by giving them props” [8†L95-L103] . For instance, in `<Greeting name="Alice" />`, the `Greeting` component gets a prop `name` with value `"Alice"`.

Inside the child component, you access props via parameters. For functional components, props come in as an object. You can destructure them:

```
function Greeting({ name }) {  
  // "name" is available here  
  return <h3>Hello, {name}!</h3>;  
}
```

Or use `props.name` if not destructuring. The above component will render `<h3>Hello, Alice!</h3>` when used as `<Greeting name="Alice" />`. You can pass any JavaScript value as a prop – strings, numbers, objects, functions, even other components [8†L95-L103] . Default HTML attributes (like `alt` or `width` on `<img>`) are also passed as props to DOM elements. Example from React docs:

```
function Avatar() {  
  return (  
      
  );  
}
```

Here `src`, `alt`, `width`, etc. are props of the `<img>` element [8†L110-L118] .

Using props to customize components: Create reusable components by passing different props. For example, a `Button` component:

```
function Button({ label, onClick }) {  
  return <button onClick={onClick}>{label}</button>;  
}
```

```

}
export default Button;

```

Now you can use `<Button label="Save" onClick={handleSave} />` or `<Button label="Cancel" onClick={handleCancel} />`. Each instance shows different text and handles clicks via the passed functions.

Default Props and Others: You can set default values or mark props optional. (This is more advanced with TypeScript or `defaultProps` property.) For simplicity, assume props are provided by the parent.

Example – Passing complex props: You can even pass objects or JSX. For instance:

```

function Profile({ person, size }) {
  return (
    <div style={{ width: size }}>
      <img src={`https://example.com/${person.imageId}.jpg`}
alt={person.name} />
      <p>{person.name}</p>
    </div>
  );
}

// Usage:
<Profile
  person={{ name: 'Lin', imageId: '1bX5QH6' }}
  size={100}
/>

```

Here `person` is an object prop and `size` is a number prop [\[8†L165-L174\]](#) [\[8†L193-L202\]](#). React docs show this exact pattern for an `Avatar` example [\[8†L165-L174\]](#).

Key points on props: Props are read-only in the child. You pass them via JSX attributes. Use destructuring to access them. They let parent components parameterize children. This enables creating mini “projects” or features by composing components with different props (e.g. a list component receiving an array prop to render items).

Mini-Example Using Props

As a simple “mini project” demonstration, imagine a **ToDoList** component that takes a list of tasks as a prop:

```

// ToDoList.js
export default function ToDoList({ tasks }) {
  return (
    <ul>
      {tasks.map(task => (
        <li key={task.id}>
          {task.title} {task.done ? "✓" : ""}
        </li>
      ))}
    </ul>
  );
}

```

```

        </li>
      )})
    </ul>
  );
}

// App.js
import TodoList from './TodoList.js';

export default function App() {
  const myTasks = [
    { id: 1, title: 'Buy groceries', done: true },
    { id: 2, title: 'Do laundry', done: false },
    // ... more tasks
  ];
  return <TodoList tasks={myTasks} />;
}

```

Here, `TodoList` receives an array of task objects via `props.tasks`. It maps over them to render an `<li>` for each. The `key` prop is used for list rendering (React best practice). This example shows how props drive the UI: changing `myTasks` changes what `TodoList` displays.

Another example: a **UserCard** that takes a `user` object prop:

```

function UserCard({ user }) {
  return (
    <div className="card">
      <h2>{user.name}</h2>
      <p>Email: {user.email}</p>
    </div>
  );
}

// Usage in App:
<UserCard user={{ name: "Alice", email: "alice@example.com" }} />

```

Props allow building flexible, reusable components. By changing the props passed in, the same component can display different content.

Citations: React docs on passing props [【8†L95-L103】](#) [【8†L165-L174】](#) .

Styling with CSS

React itself is only about structure and logic; styling is done with CSS (or CSS-in-JS, etc.). The simplest method is **plain CSS files**. In a Vite setup, you might have an `index.css` or `App.css` that you import at the top of your `main.jsx` or `App.jsx`. For example:

```

/* App.css */
.container {

```

```
padding: 1rem;
background-color: #f0f0f0;
}
```

```
// App.jsx
import './App.css';

export default function App() {
  return <div className="container">Hello styled world</div>;
}
```

As React docs note, to attach a CSS class you use `className` in JSX [【4†L209-L213】](#) . The above code applies the `.container` styles to the `<div>` .

Inline Styles: You can also apply styles directly using the `style` prop with a JavaScript object. E.g. `<div style={{ color: 'red', fontWeight: 'bold' }}>Hello</div>` . However, regular CSS or CSS Modules are more common.

CSS Modules: Vite supports CSS Modules. If you name a file `Something.module.css` , you can import styles as an object:

```
/* Button.module.css */
.primary {
  background-color: blue;
  color: white;
}
```

```
// Button.jsx
import styles from './Button.module.css';
export default function Button({ label }) {
  return <button className={styles.primary}>{label}</button>;
}
```

This scopes class names locally to avoid collisions. Vite handles CSS modules automatically when using the `.module.css` extension.

Global vs Component CSS: You can have a global stylesheet for base styles (e.g. `index.css` with body/reset styles) and component-specific CSS (either modules or imported CSS). The key is to import the CSS file so it's included in the build.

Citations: React docs on `className` usage [【4†L209-L213】](#) .

Tailwind CSS (Utility-First Framework)

What is Tailwind CSS? Tailwind CSS is a **utility-first CSS framework**. Rather than writing custom CSS, you use predefined utility classes directly in your markup. For example, Tailwind provides classes like

`flex`, `p-4`, `text-center`, `bg-blue-500`, etc., that you combine to design your UI. As Tailwind's site says: "A utility-first CSS framework packed with classes like `flex`, `pt-4`, `text-center` and `rotate-90` that can be composed to build any design, directly in your markup." [14†L13-L14] . This means you can often build an entire UI just by writing JSX with Tailwind classes, without writing separate CSS rules.

Installing Tailwind with Vite: Tailwind works seamlessly with Vite. The official Tailwind docs provide steps (using the official Vite plugin):

1. **Create a Vite project** (if you haven't already). For example: `npm create vite@latest my-project -- --template react`.
2. **Install Tailwind:** Run `npm install -D tailwindcss @tailwindcss/vite` in your project directory [15†L311-L319] .
3. **Configure the plugin:** In your `vite.config.js` (or `.ts`), add:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

This tells Vite to use the Tailwind plugin [15†L319-L327] .

1. **Create a CSS file to import Tailwind:** For example, in `src/style.css` (or `index.css`), add:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Or simply `@import "tailwindcss";` as instructed [15†L331-L338] . 5. **Include the CSS:** In your main JS entry (often `main.jsx`), import this CSS file: e.g. `import './style.css';` . 6. **Run the dev server:** `npm run dev` . Now you can use Tailwind classes in your JSX.

Tailwind will scan your JSX/HTML for class names and generate the corresponding CSS. There is *no runtime* overhead – it's all generated at build-time [15†L273-L279] .

Basic Tailwind usage: Once set up, use it like so:

```
// A Tailwind-styled card:
export default function Card() {
  return (
    <div className="max-w-sm mx-auto bg-white rounded-lg shadow-md p-6">
      <h2 className="text-xl font-bold mb-2">Card Title</h2>
      <p className="text-gray-700">This is a simple card component using
Tailwind CSS.</p>
    </div>
  )
}
```



```
);
}
```

Here, `max-w-sm` and `p-6` set width and padding, `bg-white rounded-lg shadow-md` add background and shadow, and typography classes style the text. Tailwind classes are intuitive if you learn the naming (e.g. `text-center`, `bg-blue-500`, `mt-4`, `grid`, `flex`, etc.).

Example – Responsive and Dark Mode: Tailwind also makes responsiveness and dark mode easy. You can prefix classes with breakpoints, e.g. `sm:text-lg md:text-xl` to change font size at different screen widths. For dark mode, prefix with `dark:`, e.g. `dark:bg-gray-800 dark:text-white` [【14†L113-L118】](#), to apply styles when a dark theme is active.

Key point: Tailwind is “utility-first,” meaning you rarely write CSS yourself – you compose pre-built classes. This allows rapid styling without leaving your JSX [【14†L13-L14】](#). It’s widely used for fast UI development.

Citation: Tailwind official docs (intro) [【14†L13-L14】](#); Tailwind + Vite installation guide [【15†L311-L319】](#) [【15†L331-L338】](#).

Example UI Project with Tailwind

Putting it together, let’s outline a small “UI design project” example using Tailwind. Suppose you want to build a simple blog post card:

```
export default function BlogCard() {
  return (
    <div className="max-w-md mx-auto bg-white rounded-xl shadow-md overflow-hidden md:max-w-2xl">
      <div className="md:flex">
        
        <div className="p-8">
          <h3 className="uppercase tracking-wide text-sm text-indigo-500">Blog</h3>
          <h1 className="block mt-1 text-lg leading-tight font-medium text-black">Post Title</h1>
          <p className="mt-2 text-gray-500">A short description of the post goes here. It should give a teaser of the content.</p>
          <button className="mt-4 px-4 py-2 bg-blue-500 text-white rounded hover:bg-blue-600">
            Read More
          </button>
        </div>
      </div>
    </div>
  );
}
```

This component uses Tailwind classes for layout (`flex`, `max-w-md`), typography (`text-gray-500`, `font-medium`), spacing (`p-8`, `mt-4`), and more. Notice how all styling is done via class names in JSX. In a real project, you would replace the static text and image with dynamic props or content. Tailwind speeds up UI building by reducing the need to write CSS separately.

This is just a taste – Tailwind's [documentation](#) has a comprehensive list of utility classes, plugins, and examples for various designs.

Working with Functions, Events and State

React components often include **event handler functions** and stateful logic. In functional components, you can define functions inside and pass them to elements as event handlers. For example, a button click:

```
function MyButton() {
  function handleClick() {
    alert('You clicked me!');
  }
  return (
    <button onClick={handleClick}>
      Click me
    </button>
  );
}
```

This `onClick={handleClick}` means “when the button is clicked, call `handleClick`”. Notice we pass the function reference without parentheses – React will call it when the event occurs [【17†L437-L445】](#). The Quick Start docs illustrate this pattern exactly [【17†L437-L445】](#). You can also use inline functions or arrow functions, e.g.

`<button onClick={() => setCount(count+1)}>...</button>`, though defining the function outside JSX often improves readability.

useState and updating: To let a component remember data, use the `useState` Hook. For example, a click counter:

```
import { useState } from 'react';

function CounterButton() {
  const [count, setCount] = useState(0); // initialize count to 0
  function handleClick() {
    setCount(count + 1); // update state
  }
  return (
    <button onClick={handleClick}>
      Clicked {count} times
    </button>
  );
}
```

```
);
}
```

Here, `useState(0)` returns a state variable (`count`) and a setter function (`setCount`) [17†L476-L484]. Each time the button is clicked, `handleClick` calls `setCount(count+1)`. React re-renders the component with the updated `count` value. As a result, the button text updates (“Clicked 1 times”, “Clicked 2 times”, etc.). This example directly comes from the React docs [17†L490-L498]. The docs note that after `setCount`, “React will call your component function again... this time, `count` will be 1” [17†L510-L514]. In functional components, each render has its own `count` value.

Multiple stateful components: If you render multiple `<CounterButton />`, each maintains its own state independently, as shown in the React tutorial [17†L524-L534].

Passing functions as props: Often you pass functions down to children via props. For example:

```
function Parent() {
  function handleClick(msg) {
    console.log('Child says:', msg);
  }
  return <Child onClick={handleClick} />;
}

function Child({ onClick }) {
  return <button onClick={() => onClick('Hi!')}>Send</button>;
}
```

Here the child calls `props.onClick` when the button is clicked, and the parent function receives data. This shows props and functions working together for interaction.

Summary of this section: Functions in React components are used for event handling and logic. Use `onClick`, `onChange`, etc. to listen to events, and define handler functions to respond. Manage dynamic data with hooks like `useState`. React’s reactivity ensures the UI updates when state changes.

Citation: React docs on responding to events and using state [17†L437-L445] [17†L476-L484].

Interview Questions & Answers

1. What is React and why use it?

Answer: React is a JavaScript library for building user interfaces [21†L232-L240]. It uses a component-based approach to compose UIs and a virtual DOM for efficient updates [7†L155-L163] [21†L243-L247]. It allows declarative code: describe what the UI should look like, and React handles rendering. It’s chosen for its speed, modularity, and large ecosystem.

2. What is JSX?

Answer: JSX is a syntax extension for JavaScript that looks like HTML. It’s used in React to describe UI. You write elements like `<div>Hello</div>` in your JS code. Under the hood, it’s compiled

to `React.createElement` calls. JSX is stricter than HTML (e.g. you must close tags and wrap multiple siblings in one parent) [4†L179-L187] .

3. Explain Virtual DOM vs Real DOM.

Answer: The Real DOM is the browser's live representation of the HTML. Manipulating it directly can be slow for large apps. The Virtual DOM is React's in-memory copy of the UI. When state changes, React diffs the new virtual DOM against the old one and updates only what's changed in the real DOM [7†L155-L163] . This results in faster, batched updates and smoother UI performance.

4. What are React components?

Answer: Components are reusable pieces of UI (as functions or classes). A functional component is a JavaScript function that returns JSX [4†L109-L118] . Components can accept **props** as input and return UI. They allow splitting the UI into independent, reusable parts. For example:

```
function MyComponent() { return <div>...</div>; }
```

5. How do you pass data between components?

Answer: You pass data from parent to child via **props**. Props are like function parameters. A parent uses a child component as a JSX tag and adds attributes. For example: `<Greeting name="Alice" />`. Inside `Greeting`, access `name` via `function Greeting({ name }) { ... }` [8†L95-L103] . Data flows down: parent → child. (To share data between siblings or non-parent-child, you lift state up or use Context/redux, etc.)

6. State vs Props – what's the difference?

Answer: Props (short for "properties") are inputs passed to a component by its parent and are read-only inside the component [8†L95-L103] . **State** is data managed inside a component (using `useState` in functional components). State is internal and can be changed by the component (e.g. via `setState`), causing re-render. Props come from outside; state lives inside.

7. How do you add styles in React?

Answer: Several ways. Commonly, you create a CSS file and import it (e.g. `import './App.css'`), then use `className="..."` in JSX to apply classes [4†L209-L213] . For example `<div className="container">`. You can also use inline styles (e.g. `<div style={{ margin: 10 }} />`), CSS modules (`import styles from './file.module.css'`), or libraries (like styled-components). Importantly, in JSX the attribute is `className`, not `class` [4†L209-L213] .

8. What is Tailwind CSS and why use it?

Answer: Tailwind CSS is a utility-first CSS framework. It provides low-level utility classes (like `p-4`, `text-center`, `bg-blue-500`) that you compose in your HTML/JSX to build designs [14†L13-L14] . Instead of writing custom CSS, you mix classes in your markup. It speeds up development and ensures consistency. To use it in React with Vite, install `tailwindcss` and configure it as a plugin [15†L311-L319] .

9. How do you handle events in React?

Answer: You attach event handler functions to elements via props like `onClick`, `onChange`, etc. For example: `<button onClick={handleClick}>`. The value should be a function reference. When the event happens, React calls that function [17†L437-L445] . For example:

```
function handleClick() { alert('Clicked!'); }  
<button onClick={handleClick}>Click</button>
```

Notice we do not write `handleClick()` with parentheses, otherwise it would run immediately. React calls it only on the click.

1. What is `useState` and how do you use it?

Answer: `useState` is a React Hook that adds state to functional components. You import it with `import { useState } from 'react'`, then call it inside your component. It returns a state value and a setter function: `[value, setValue] = useState(initialValue)`. For example:

```
const [count, setCount] = useState(0);
```

Initially, `count` is `0`. To update, call `setCount(newCount)`. React will re-render the component with the new state `【17†L476-L484】`. This is how you make dynamic, interactive components (e.g., counters, form inputs, etc.).

2. Explain the `key` prop in lists.

Answer: When rendering lists (`array.map(...)`), React requires a `key` prop on each element to identify it uniquely. This helps React optimize re-renders by tracking which items changed. Typically, use a unique ID from your data. For example:

```
{items.map(item => <li key={item.id}>{item.name}</li>)}
```

Never use indices as keys if items can reorder.

3. What is the difference between a framework and a library? (general)

Answer: A library (like React) offers tools and components you call/invoke in your code. A framework enforces more structure and calls your code (inversion of control). React is often called a library because you integrate it into your app as needed, and it doesn't force a rigid architecture. (Reference context: MDN notes React is "not a framework – it's not even exclusive to the web." `【21†L232-L240】`).

These questions cover key points for a React developer. By combining official docs and examples, you can answer them confidently.
